

39. Plánování a synchronizace procesů, transakce

39. Plánování a synchronizace procesů

SZZ okruh 39 (FIT BUT). Souběžné procesy — kdo běží a jak se nepošlapou.

Shrnutí

Procesy a vlákna

- **Proces** = běžící program (PID, stav, registry, zásobník, data, zdroje); **PCB** ho reprezentuje.
- **Vlákno** = odlehčený proces; vlastní registry/zásobník, **sdílí** kód/data/zdroje → rychlejší vznik a přepínání.
- Tvorba: **fork** (duplikace, copy-on-write), **exec** (změna programu); předek init (PID 1).

Plánování

- **Nepreemptivní** (proces předá řízení sám) × **preemptivní** (jádro vynutí přerušením).
- Algoritmy: **FCFS** (FIFO), **Round Robin** (časové kvantum), **SJF/SRT** (nejkratší úloha, riziko stárnutí), víceúrovňové (se zpětnou vazbou), **CFS** (férový, RB-strom virtuálního času).
- **Inverze priorit** (řeší dědění priority); přepnutí kontextu řídí **dispatcher**.

Synchronizace

- **Race condition / data race** při souběžném přístupu ke sdíleným zdrojům → **kritická sekce** (vzájemné vyloučení).
- **Spinlock** (aktivní čekání; Peterson, bakery, atomická SWAP), **semafor** (pasivní čekání, fronta), **mutex** (binární semafor), **monitor** (wait/notify).
- **Deadlock** — cyklické čekání; **Coffmanovy podmínky** (vzájemné vyloučení, hold-and-wait, neodnímatelnost, cyklická závislost); řešení: prevence / vyhýbání / detekce + zotavení. Dále **starvation**, **livelock**.

Paměť/IPC viz [[okruhy/38-sprava-souboru-pameti]]; přerušení viz [[okruhy/06-periferie-preruseni-dma-sbernice]]; transakce/ACID viz [[okruhy/36-relacni-data-sql-transakce]].

Související syntéza

- [[synthesis/preruseni-udalosti-signaly | Přerušení × události × signály]] — syntéza
- [[synthesis/zasobnik-napric-obory | Zásobník napříč obory]] — syntéza
- [[synthesis/transakce-acid-db-os | Transakce a ACID: DB × OS]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Procesy a vlákna □ [[#Procesy a vlákna]] - *Proces vs. vlákno?* □ Proces má vlastní paměť; vlákna sdílí kód/data procesu (jen vlastní registry/zásobník), rychlejší přepínání. - *Stavy procesu?* □ init, runnable, running, waiting, terminated (zombie).

Plánování □ [[#Plánování]] - *Preemptivní vs. nepreemptivní?* □ Jádro může přerušit (RR, CFS) × proces běží, dokud sám nepředá řízení (FCFS, SJF). - *FCFS/RR/SJF?* □ FIFO; RR časové kvantum; SJF nejkratší úloha (riziko stárnutí).

Synchronizace □ [[#Synchronizace]] - *Kritická sekce, vzájemné vyloučení?* □ Úsek se sdíleným zdrojem; v daném okamžiku v ní max. jeden proces. - *Semafor vs. mutex?* □ Semafor = čítač + fronta (pasivní čekání); mutex = binární semafor (vstup jednomu). - *Deadlock a Coffmanovy podmínky?* □ Cyklické čekání; 4 podmínky (vzájemné vyloučení, hold-and-wait, neodnímatelnost, cyklus) — porušením jedné se předchází.

Plné znění (ke studiu)

Správa procesů zahrnuje:

- **přepínání kontextu** (dispatcher): fyzické **odebírání a přidělování procesoru** procesům na základě rozhodnutí plánovače,
- **plánovač** (scheduler): rozhoduje, který proces poběží a případně jak dlouho.
- **správu paměti** (memory management): přidělování paměti procesům,
- **meziprocovou komunikaci** (IPC; Inter-process communication): signály, sdílená paměť, roury, sockety, ...

Proces

Jedná se o běžící program. V OS je identifikován a náleží mu:

- **identifikátor - PID** (process identifier),
- **stav plánování** (běžící, pozastavený, čekající, připravený běžet, ukončený (zombie), ...),
- **program**, který jej řídí,
- **obsah všech registrů**,
- **zásobník** - rozpracované funkce,
- **data** - statická, hromada,
- **zdroje OS** - otevřené soubory, semafore, sdílená paměť, ... [[media/szz-39/media/image2.png]]

[[media/szz-39/media/image1.png]]

Process Control Block (PCB)

Je datová struktura, pomocí které je v OS **reprezentován proces** (někdy také control block nebo task struct). PCB zahrnuje (případně odkazuje na):

- **identifikátory** spojené s procesem (PID),
- **stav plánování** procesu (běžící, pozastavený, ...),
- **obsah registrů** (včetně EIP a ESP),
- **plánovací informace** (priorita),
- informace spojené se **správou paměti** (tabulky stránek, ...),

- Informace spojené s **účtováním** (doba využití procesoru, ...),
- **Využití I/O zdrojů** (otevřené soubory, používaná zařízení, ...).

Části procesu v paměti v Unix

- **Uživatelský adresový prostor** přístupný procesu (kód, hromada, statická data, ...).
- **Uživatelská oblast** uložená pro každý proces spolu s **daty, kódem a zásobníkem** a **částí PCB** (PID, obsah registrů, fd, obslužné funkce pro signály, účtování, pracovní adresář, ...) v uživatelském adresovém prostoru, je **přístupná pouze jádru**.
- **Záznam v tabulce procesů**, který obsahuje informace o procesu, které jsou důležité, i když zrovna neběží (PID, stav plánování, událost, na kterou čeká, čekající signály, ...). Je **trvale uložen v jádře**.

Kontext procesu

- **uživatelský kontext**: kód, data, zásobník, sdílená data,
- **registrový kontext**: obsah registrů,
- **systémový kontext**: systémové údaje o procesu tj. záznam v tabulce procesů, tabulka stránek, procesem otevřené soubory, ...

Tvorba procesu

Nový proces vytváříme **duplikací aktuálního** pomocí funkce **fork** (vrací 0 v child procesu a PID potomka v rodičovském procesu). Duplikovaný proces je **takřka totožný**. Uživatelský adresový prostor má **stejný** obsah jako obsah rodiče, sdílí otevřené soubory, obsluhu signálu, ... Pro omezení plýtvání pamětí se používá **copy-on-write** (data se doopravdy duplikují, až když jeden z procesů začne zapisovat). Potomkovský proces se **liší** návratovou hodnotou fork (0), identifikátory, údaji o plánování a účtování, nedědí čekající signály a zámky souborů. Předkem všech uživatelských procesů je proces **init s PID=1**, tento proces přebírá návratový kód (tím je proces ukončen ze stavu zombie/mátoha) i procesů, kterým dřív skončí rodič.

Změna vykonávaného programu se provádí pomocí **exec()**.

Vlákna

Jedná se o odlehčený proces, v rámci jednoho procesu jich může běžet více. Program je ve vláknech vykonáván paralelně. Vlákna mají své **vlastní registry a zásobník, sdílí kód, data a další** zdroje (otevřené soubory, signály, tabulku stránek). Vlákna se **rychleji vytvářejí** a je **rychlejší** mezi nimi **přepínat**. Komunikace mezi vlákny je také jednodušší, protože sdílí data (nutno používat obezřetně - uzamykat). Pro paralelizaci úloh se běžně používají vlákna místo procesů.

Plánování procesu

[[media/szz-39/media/image4.png]]

Plánovač procesů (**scheduler**) rozhoduje, **který proces poběží** a případně **jak dlouho**. Existují dva druhy plánování:

- **Nepreemptivní plánování**: změnu procesu lze **provést pouze**, pokud aktuálně běžící proces **předá řízení jádru** (např. I/O operace, exit, yield), tj. dokud chce běžet, nic jej nemůže přerušit.

- **Preemptivní plánování:** změnu procesu může **jádro vynutit**, i když mu nepředá řízení, vyvoláním **přerušeni** (typicky od časovače, ale třeba i od disku, aj.).

Samotné **přepnutí** procesu **řídí dispečer (dispatcher)**. Plánování závisí na **prioritě** procesu a **dostupnosti zdrojů**, které potřebuje (např. paměti, dokončení IO operace, otevírání souboru, ...). **Spuštění** nových procesů se také plánuje a může být **pozdrženo** (z důvodu nedostatku paměti).

Přepnutí procesu (kontextu)

Přepnutí kontextu provádí dispečer a znamená odebrání procesoru procesu A jeho přidělení procesu B, což zahrnuje:

1. **Úschovu stavů a registrů** procesu **A** do **jeho PCB**.
2. **Úprava** některých řídicích **struktur v jádře** a třeba i zrušení záznamů procesu **A** v **TLB** (záleží na implementaci TLB).
3. **Obnova registrů** procesu **B** z **PCB**.
4. **Předání řízení** na adresu **procesu B**, odkud bylo dříve přerušeno.

Neukládá se a neobnovuje se **celý stav** procesu. Např. namapované stránky na rámce v paměti zůstávají, **mění se pouze ukazatel na tabulku stránek**, se kterou se bude aktuálně pracovat.

Přepnutí přesto trvá několik **stovek** nebo **tisíc instrukcí**, proto doba mezi přepínáním musí být dostatečně dlouhá, aby procesor netrávil nepřiměřeně dlouho přepínáním kontextu.

Plánovací algoritmy

Algoritmy pro plánování mohou upřednostňovat některý z požadavků na plánování procesu:

- Využití CPU,
- Propustnost,
- Doba běhu,
- Doba čekání,
- Doba odezvy.

First Come, First Served (FCFS)

Procesy čekají na přidělení procesoru ve frontě **FIFO**. Jedná se o **nepreemptivní algoritmus** (tzn. proces se musí sám vzdát procesoru žádostí jádra nebo voláním yield). Procesor je **přidělen** procesu na **začátku fronty** a proces, kterému je procesor **odebrán**, je zařazen **na konec fronty**, stejně jako nový proces.

Round Robin

Algoritmus založený na principu **FIFO** jako FCFS, je ale preemptivní, tzn. procesům je přiděleno **časové kvantum** (doba kterou mohou strávit výpočtem, než jim bude CPU odebráno), po **vypršení kvanta** nebo pokud se proces **vzdá** procesoru (yield nebo žádostí jádra např. o I/O) je zařazen na konec fronty a procesor je přidělen procesu na začátku fronty.

Shortest Job First (SJF)

Algoritmus přiděluje procesor procesu, který požaduje **nejkratší dobu** pro své další provádění na procesoru **bez I/O operací**. Jedná se o **nepreemptivní** algoritmus. **Minimalizuje** průměrnou **dobu čekání** a **zvyšuje propustnost** systému, OS ale musí znát nastávající dobu procesu nebo jí aspoň dobře odhadnout. Používá se proto ve **specializovaných** systémech, kde se **opakovaně provádí podobné** úlohy. Má ale jeden zásadní problém - **stárnutí** (starvation, vyhladovění): procesy s delším požadavkem na CPU se nemusí nikdy k CPU dostat.

Shortest Remaining Time (SRT)

Jedná se o obdobu algoritmu **SJF**. Algoritmus je **preemptivní**, ale tím, že procesor aktuálně používá proces, který má **nejmenší dobu pro dokončení**, může k preempci dojít, pouze když dojde k **vytvoření** nového procesu s **kratším časem dokončení**, než je čas dokončení aktuálně běžícího procesu.

Víceúrovňové plánování

Procesy jsou rozděleny do různých **skupin** (typicky dle **priority**). V rámci každé **skupiny** lze použít **jiný plánovací algoritmus**. Dále existuje **další algoritmus**, který **určuje** (nejčastěji na základě priority), jaké skupině umožní výběr procesu (dle algoritmu dané skupiny). Může docházet ke **hladovění** a **inverzi priorit**.

Víceúrovňové plánování se zpětnou vazbou

Procesy jsou rozděleny do **skupin dle priorit**. Plánování může proběhnout jednou z variant:

- Nově připravené procesy běžet jsou zařazovány do fronty s **největší prioritou** (používá se **Round Robin** - preemptivní). Jejich **priorita postupně klesá** a tím se přesouvají do front nižší úrovně, až nakonec jsou plánovány v nejméně prioritní frontě (tam se dostanou především dlouho trvající procesy).
- Proces je po spuštění zařazen do prioritní fronty dle jeho stanovené počáteční priority (statická priorita). Následně se jeho priorita mění (dynamická priorita):
 - **roste**, pokud proces často **čeká na I/O** (asi jej uživatel hodně používá, takže zajistíme jeho rychlou reakci),
 - **klesá**, pokud proces **spotřebovává hodně** procesorového času.

Completely Fair Scheduler (CFS)

Přiděluje procesor každému procesu tak, aby mu bylo přiděleno **odpovídající procento procesorového času dle jeho priority**. U každého procesu si tak musí vést údaj o **stráveném virtuálním** procesorovém čase (vliv priority) a procesor přiděluje procesu s **nejmenším** stráveným virtuálním časem. Procesy organizuje dle **stráveného času** (nejméně v kořeni) do **Red-Black Tree**. Nově vzniklým procesům musí být přidělen nějaký počáteční virtuální čas dle jeho priority (protože běžící procesy již strávili čas na CPU). Algoritmus je **preemptivní** a časové **kvantum** je procesu vypočteno **na základě priority** (respektive méně prioritním procesům ubíhá čas rychleji než prioritním). Po přepnutí kontextu je nově proces, jemuž byl odebrán procesor, zpět vložen na **odpovídající místo v RB-Tree**. Algoritmus také umožňuje plánovat procesy **více uživatelům** nebo z **více terminálů** s různou prioritou.

Plánovače v Linux a Windows

Procesy jsou rozděleny do několika (desítky) úrovní priority. **Nejvyšší** priority mají procesy **reálného času** (plánovány pomocí Round Robin), **střední** prioritu mají **běžné procesy** (plánovány po-

mocí CFS) a **nejnižší** prioritu mají **idle** procesy (většinou zaměstnávají CPU, když není co na práci). Procesy jsou startovány se **statickou prioritou**. Dynamická priorita se poté může měnit na základě chování procesu (zejména na Win):

- **Priorita se zvyšuje:** okno procesu je na **popředí**, do okna procesu **přichází zprávy** (myš, klávesnice, ...), proces je **uvolněn z čekání** na I/O.
- **Priorita klesá:** Proces spotřebovává moc CPU času, po vyčerpání každého kvanta.

Inverze priorit

Je stav, kdy efektivně priorita nízko-prioritního procesu přesáhne prioritu vysoko-prioritního procesu. Stane se tak, pokud **nízko-prioritní proces obsadí nějaký zdroj** a je mu odebrán CPU, než jej uvolní. Následně některý z **vysoce prioritních procesů potřebují tento zdroj**, ale musí na něj čekat. Nízko-prioritní proces mezi tím **předbíhají jiné procesy** s vysokou a střední prioritou. Vysoce-prioritní proces se tak dostane na řadu, až za dlouhou dobu (nejdříve musí být uvolněn požadovaný zdroj). Inverze priorit nemusí být problém, zvyšuje však obvykle odezvu systému a v případě **real-time procesů** pracujících např. s HW může být **kritická**. Inverzi priorit lze řešit:

- procesům v **kritické sekci** je přiřazena **největší priorita**,
- procesy, na které **čeká jiný proces**, **dědí jeho prioritu** (pokud je vyšší),
- **zákaz přerušování** pokud je proces **v kritické sekci**.

Komplikace plánování procesů

- **víceprocesorové systémy** (dnes snad všechny) vyžadují **vyvažování** výkonu jednotlivých CPU (jader), zde je ale navíc **problém s obsahem cache** atd.
- **hard real-time systémy**, u kterých musí plánovač zajistit (**garantovat**) určitou **odezvu**.

Komunikace mezi procesy

IPC = Inter-Process Communication, provádí se pomocí:

signály (kill, signal, ...),

roury (pipe, mkfifo, ...),

zprávy (msgget, msgsnd, msgrcv, msgctl, ...),

sdílená paměť (shmget, shmat, ...),

sockety (socket, ...).

Procesy jsou členěny na:

- **Úloha** - Skupina paralelně běžících procesů spuštěných jedním příkazem a propojených do **pipeline** (pomocí rour: \$.x|.y|.z).
- **Skupina procesů** - Množina, které je množné **poslat signál jako jedné jednotce** (může mít vedoucího - tím je implicitně první proces), child procesy jsou přidávány do stejné skupiny jako rodič, lze ale změnit.
- **Sezení** - Každá skupina procesů je **v jednom sezení** (může mít vedoucího).

Signály

Jeden z principů komunikace mezi procesy. Jedná se o **číslo (int)**, které je zasláno procesu pomocí **speciálního rozhraní**, které je pro to určeno. Lze zde najít **analogii** s HW přerušením pro SW úroveň. Také se **dějí asynchronně** a také pro ně **definujeme obslužné funkce**. Signály jsou generovány při:

- chybách (chybná práce s pamětí, aritmetická chyba - dělení 0, ...),
- externích událostech (vypršení časovače, dokončení I/O, ...),
- na žádost procesu (např. rodič posílá signál potomkovi, signál kill, ...).

Obsluha signálů může být **složitá** a **potenciálně obsahovat chyby** (kvůli asynchronnímu vzniku signálů), při obsluze **nelze použít některé funkce**, protože by to mohlo ovlivnit normální běh procesu např. **printf()**. Pokud vzniknou dva signály hned za sebou, může být druhým přerušena obsluha prvního. Příklady: SIGHUP, SIGINT, SIGKILL, SIGALARM, SIGSTOP, SIGCONT, SIGCHLD, SIGUSR1, SIGUSR2, ... Kromě **SIGKILL** a **SIGSTOP** lze signály předefinovat. Implicitní reakce na signály jsou buď ukončení procesu (časté), ignorování signálu, zmrazení procesu, nebo rozmrazení procesu. Signály až na SIGKILL, SIGSTOP, SIGCONT lze blokovat. Nastavení **obsluhy signálů** včetně blokování se **dědí** na potomky **při fork**. Při **exec** se nastaví **implicitní obsluha**. Signály se **zasílají** (funkce kill) na základě **znalosti PID** případně lze zaslat všem procesům určité skupiny (např. potomkům). Čekání na signály pomocí pause() nebo sigsuspend().

Synchronizace procesů

Synchronizaci provádíme, aby při **paralelním přístupu ke sdíleným zdrojům** (sdílená paměť, soubory, I/O zařízení, ...) **nedocházelo k chybám způsobených nesprávným pořadím** provedených dílčích **operací** různými procesy. Např. 2 procesy sdílí paměť:

proces 1 proces 2

kód:

N++; N--;

assembler:

register = N register = N

register = register + 1 register = register - 1

N = register N = register

Pokud je **N** na začátku **1**, mělo by být **1** i po provedení následujícího kódu (oběma procesy), ale může být také **0** nebo **2**.

Race condition

Časově závislá chyba (souběh) je chyba, která vzniká při **přístupu ke sdíleným zdrojům** kvůli **různému** pořadí provádění jednotlivých **paralelních výpočtů** v systému.

(Sdílená) kritická sekce

Jedná se o **úseky v programech**, které řídí **paralelně** běžící procesy, ve kterých dochází k přístupu ke **sdíleným zdrojům**. Provádění jednoho z **úseků** jedním procesem **vylučuje** provádění **libovolného z těchto úseků** ostatními procesy.

Problém kritické sekce

Jedná se o problém zajištění **korektní synchronizace** procesů na množině **sdílených kritických sekcí**. Je nutné **zajistit**:

- **vzájemné vyloučení**: nanejvýš **jeden** proces je v daném **okamžiku** v dané **množině kritických sekcí**. Případně je tam nanejvýš **k** procesů.
- **dostupnost kritické sekce**: Je-li kritická sekce **volná**, respektive je volná aspoň v určitých okamžicích, proces **nemůže čekat neomezeně dlouho** na **přístup** do ní.

Musíme se **vyhnout**:

- uvážnutí (deadlock),
- blokování (blocking),
- stárnutí (hladovění - starvation).

Data race

Časově závislá chyba nad daty nebo také souběh nad daty. K této chybě může dojít, pokud ke zdroji s **výlučným** přístupem přistupuje **více procesů bez synchronizace** (2 a více) a alespoň **jeden** k němu přistupuje **pro zápis**.

Deadlock (uvážnutí)

Jedná se o situaci, kdy **každý proces** z určité **neprázdné** množiny procesů je **pozastaven a čeká** na uvolnění nějakého zdroje s **výlučným přístupem**, který je **vlastněn** nějakým procesem z dané množiny, který **jediný jej může uvolnit**, a to až po dokončení práce s ním. (Ale tu dokončit nemůže, protože čeká. Takže každý proces čeká na jiný proces - v **grafu** čekání se vytvořila **kružnice**).

Podmínky uvážnutí (Coffmanovy podmínky)

1. **Vzájemné vyloučení** při použití **prostředků** (tj. k prostředku může v jeden čas přistupovat pouze jeden proces),
2. **vlastnictví** alespoň **jednoho** prostředku (zdroje), pozastavení **a čekání** na další (tj. proces má alokovaný nějaký zdroj, potřebuje i jiný, o něj žádá, ten není ale k dispozici, tak čeká),

prostředky **vrací proces**, který je **vlastní**, a to až po dokončení jejich používání (tj. pokud má proces alokovaný nějaký zdroj, např. I/O zařízení, pouze tento proces může říct, že už jej nepotřebuje, a to až v moment, kdy jej doopravdy nepotřebuje - dokončil čtení/zápis),

3. **cyklická závislost** na sebe čekajících procesů (tj. proces A čeká na proces B a proces B čeká současně na proces A).

Řešení uvážnutí

- **prevence uvážnutí**: porušení alespoň jedné z **Coffmanových podmínek**,
- **vyhýbání se uvážnutí**: kontrola grafu **žádostí** a **vlastnictví zdrojů**, zamezení vzniku kružnice.
- **detekce a zotavení**: existuje speciální proces (mimo množinu procesů, na které došlo k uvážnutí), který detekuje vznik uvážnutí a nějakým způsobem jej vyřeší.

Blocking (blokování)

Blokování při přístupu do kritické sekce je situace, kdy proces, který žádá o přístup do kritické sekce, **musí čekat**, i když je **kritická sekce volná** (žádný proces se nenachází v žádné z množiny těchto kritických sekcí - jinak řečeno, **žádný proces nepoužívá daný sdílený zdroj s výlučným přístupem**).

Starvation (stárnutí)

Stárnutí je situace, kdy **proces čeká** na splnění podmínky, která **nemusí nikdy nastat**. V případě kritické sekce se jedná o splnění podmínky, která umožňuje vstup do kritické sekce. V případě plánování procesů se jedná o situaci, kdy proces nemusí být nikdy plánovačem naplánován pro běh na procesoru (předbíhají jej jiné procesy). Uváznutí (deadlock) i blokování (blocking) lze zobecnit na stárnutí.

Livelock

Jedná se o zvláštní situaci stárnutí v kombinaci s uváznutím. Každý proces z určité neprázdné množiny **běží** ale provádí jen **omezený úsek kódu**, ve kterém **opakovaně** žádá o přístup do **kritické sekce** (žádá o zdroj s výlučným přístupem), ve které je jiný proces z dané množiny procesů, který **jediný jí může uvolnit**, pokud by mohl pokračovat.

Zajištění výlučného přístupu do KS - synchronizace

Výlučný přístup do KS se zajišťuje pomocí tzv. **spinlock**. Jedná se o řešení, které umožňuje **aktivní čekání**. Spinlock lze implementovat pomocí:

- **speciálních algoritmů**, které nevyžadují atomické operace. Např.
 - **Petersonův algoritmus**: Proces nastavením příznaku vyjádří svůj zájem o kritickou sekci, ale zároveň dá přednost jinému procesu. Následně čeká na uvolnění kritické sekce, nebo až mu druhý proces vrátí přednost.
 - **Bakery algoritmus L. Lamporta**: Před vstupem do KS získá proces **lístek**, jehož hodnota je větší než hodnota lístků procesů, které již lístek mají a čekají. Pokud je hodnota stejná (kvůli tomu že lístek procesy získaly současně - souběžně četly největší hodnotu), rozhoduje se podle velikosti **PID**. Držitel **nejmenšího čísla** lístku **může vstoupit** do kritické sekce,
- **pomocí atomických instrukcí**, jejíž atomicitu zajišťuje HW implementace, např. instrukce **SWAP**.

Aktivní čekání lze využít na krátkých neblokujících sekcích bez preempce (tj. zákazem přerušování). Tyto krátké sekce se mohou nacházet např. v **implementaci operací nad semaforem** nebo **monitorem**, **nikoliv** však v uživatelském kódu. Klasickými synchronizačními problémy jsou např. **producent a konzument** (producent zapisuje do fronty, konzument odebírá), **čtenáři a písáři** (současně může číst libovolný počet čtenářů, při zápisu však může přistupovat pouze písáři).

Semafor

Synchronizační nástroj, který **minimalizuje** (případně úplně odstraňuje) **aktivní čekání**. Pokud proces aktuálně nemůže vstoupit do kritické sekce (tj. uzamknout semafor - uzamčení musí být **atomické**, zde se vyskytuje **spinlock** a **zákaz přerušování**), je **pozastaven** (nečeká aktivně) a umístěn do **fronty čekajících procesů** (pořadí procesů není garantováno) na uvolnění KS. Obecně mohou být semaforey implementovány pomocí **celočíselné proměnné** a **fronty**. Proměnná udává, kolik

procesů může ještě vstoupit do KS. Pokud je **0 a menší** sekce je **uzamčena**. Zvláštním (a nejpoužívanějším) případem je poté **binární semafor - mutex**, který umožňuje vstup do KS pouze **jednomu procesu**, který jediný jej může zpět odemknout.

Monitor

Jedná se o vysokoúrovňový synchronizační prostředek. Zapouzdřuje data a poskytuje operace, které zajišťují správné provádění operací nad chráněnými daty.

- **wait**: uspí dané vlákno/proces (pasivní čekání) a zařadí jej do fronty čekání na nějaké podmínce, proces se tímto vzdává zámku (musí jej předtím vlastnit),
- **notify/signal**: pro danou podmínku **probudí jeden** z čekajících procesů (pokud nějaký existuje, jinak prázdná operace) na danou podmínku, které dříve volaly wait. (Operace taky může odebrat zámek procesu, který ji volá, a přidělit jej probuzenému procesu.)
- **notifyAll**: **probudí všechny** čekající procesy (pokud nějaké čekají, jinak prázdná operace).

Prevence uváznutí (podrobněji)

Zrušíme platnost některé z **nutných podmínek** uváznutí (**Coffmanovy podmínky**). To lze zabudovat přímo do celého implementovaného systému a nějak ověřit (verifikace), nebo pro tento účel využívat systém přidělování zdrojů, který bude vždy vynucovat porušení alespoň jedné podmínky. Příklady řešení pro jednotlivé podmínky:

1. **Nepoužíváme** sdílené prostředky nebo používáme takové, ke kterým **není** nutné přistupovat **vylučně** (může k nim současně přistupovat více procesů).
2. **Statická** (tj. program neběží) kontrola kódu, že proces žádá o zdroj (a tedy může být pozastaven), jen v případě, že aktuálně **žádný zdroj nevlastní**. Nebo **dynamická** kontrola uvedeného, pak je ale nutné řešit, co s takovým procesem. (Nabízí se jeho ukončení a odebrání jím vlastněných zdrojů, to ale může vést na nekonzistence a nedokončení některých úloh).
3. Při pozastavení procesu (neúspěšné žádosti o další zdroj) jsou mu **odebrány všechny jím vlastněné** zdroje. Ty jsou mu **vráceny** včetně nově žádaného zdroje, **až jsou všechny k dispozici**, a proces je znovu spuštěn. (Odebrání zdrojů v průběhu práce s nimi může způsobit nekonzistence).
4. **Očíslování prostředků** a jejich přidělování v **pořadí**, které **vylučuje** vznik cyklické závislosti (tj. **vzestupně** nebo **sestupně**).

Vyhýbání se uváznutí (podrobněji)

Obecně lze problém řešit tak, že procesy **deklarují**, jaké zdroje **aktuálně potřebují** a na základě aktuálního **stavu přidělených** zdrojů a těchto **žádostí o zdroje** se rozhodne, jestli **může** být dané žádosti **vyhověno**, nebo proces **musí čekat**. Žádosti o zdroj je vyhověno, pokud aktuálně **nemůže vzniknout cyklická** závislost na sebe čekajících procesů a **cyklická závislost nebude možná ani v budoucnu**, a to při zohlednění té **nejhorší možné** situace.

V praxi se tento problém řeší pomocí grafových algoritmu, konkrétně pomocí **grafu alokace zdrojů**. Zdroj je **přidělen** jen tehdy, pokud **nehrozí vznik cyklu v grafu**, jinak proces musí čekat. (Vznik cyklu v grafu neznamená vznik uváznutí, pouze značí, že k uváznutí může dojít).
[[media/szz-39/media/image3.png]]

Detekce uváznutí (podrobněji)

Na dané **neprázdne množině** procesů může dojít k **uváznutí**, ale existuje nějaký **další proces mimo tuto množinu**, který periodicky (nebo nějak jinak) **kontroluje**, zda k uváznutí **došlo**, a pokud ano, **provede zotavení** (zajistí, že uváznutí bude zrušeno).

Detekce uváznutí může být v konkrétním případě řešena **pomocí grafu vlastnictví a čekání na zdroje**. **Cyklus** v tomto grafu znamená vznik **uváznutí** (skutečný vznik, ne pouze možnost vzniku jako v grafu žádostí).

Zotavení z uváznutí lze nejjednodušeji řešit **ukončením** (restartem) některého z procesů - **victim** (děje se tak např. u databázi v SŘBD) a případně doplnit ukončení procesu o **rollback** (viz transakce v DB). Dále lze zotavení řešit **odebráním zdrojů** některému procesu a jejich **přidělení jinému** a poté zajištění, že tomuto procesu budou opět přiděleny včetně těch, které si chtěl nově zabrat.

Transakce na úrovni OS

Jedná se o druh zpracování, při kterém je **skupina logických operací - transakce**, provedena jako **jeden celek**, tj. buď **vůbec nebo všechny**. Pokud se při zpracování v rámci transakce vyskytne jakákoliv **chyba** a transakce nemůže být dokončena, všechny dílčí operace musejí být **vráceny do stavu před začátkem transakce**. Jejich zpracování se musí **vypořádat s výskytem poruch a paralelismem**. Definice z databází pomocí **ACID** zde nemusí být úplně přesná. Např. dosažení **trvalosti** (durability) nemusí být možné, pokud za transakci považujeme přepnutí kontextu (asi těžko zajistíme, že po dokončení a přepnutí kontextu a následném pádu systému bude přepnutí kontextu trvalé). U transakcí na úrovni OS tak zajišťujeme zejména **A** (atomicity - atomicitu), tedy že transakce je provedena v celém svém rozsahu, nebo není provedena žádná z jejích částí (nemohou být při přepnutí kontextu ponechány stejné registry a pouze změněn zásobník).

Typickým **příkladem** transakčního zpracování na úrovni OS je **zápis na disk**. Využívá se zápisu do **žurnálu** a řešení chyb pomocí **REDO** a **UNDO**, tedy velmi podobný princip databázové transakci. Těžko ale v tomto případě zajistíme **C** (consistenci, konzistenci) zapsaných dat **ve smyslu jejich významu**, můžeme pouze zajistit, že bity jsou zapsané přesně tak, jak vyžaduje proces.

Zdroje

- SZZ okruh 39 — studijní materiály FIT BUT (szz-39.docx). Obrázky: media/szz-39/.

Navigace: [[index | • Přehled okruhů]] • [[okruhy/38-sprava-souboru-pameti | 38. Správa souborů a paměti]] • další: [[okruhy/40-objektova-orientace | 40. Objektová orientace]]